PUCE

# Programación Móvil

## Tema 5

# Introducción

El Tema 5 marca un **cambio estructural** en la forma de construir interfaces móviles. Dejamos atrás el sistema de vistas imperativo (XML) para adentrarnos en **Jetpack Compose**.

Este enfoque elimina la "desincronización entre el estado lógico de la aplicación y lo que el usuario visualiza en pantalla".

*"En la API 36, dominar Compose es un requisito mandatorio para desarrollar aplicaciones resilientes y alineadas con los estándares de la industria".*

XML COMPOSE



# Objetivos

### Arquitectura de Paradigmas

Diferenciar técnica y conceptualmente el paradigma imperativo (XML) del declarativo (Compose).

### Ciclo de Vida & Recomposición

Comprender el ciclo de vida de un Composable: Composición, Recomposición y Árbol de Composición.

### Convergencia Multiplataforma

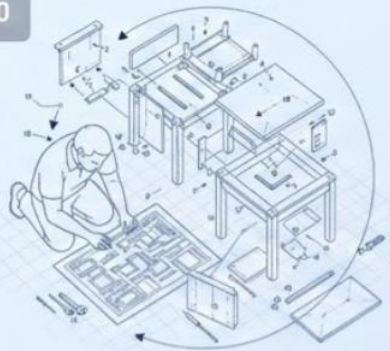
Identificar similitudes arquitectónicas entre Jetpack Compose y SwiftUI para Android/iOS.

### Reactividad en API 36

Implementar interfaces reactivas que se actualicen automáticamente ante cambios de estado.

# Cómo vs Qué: La Diferencia Filosófica

IMPERATIVO



## El enfoque del "Cómo"

Describe la secuencia exacta: **"Camina 5 pasos, gira a la derecha..."**. Frágil ante cambios del entorno.

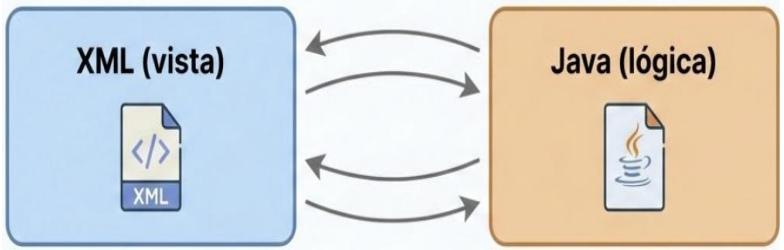
DECLARATIVO

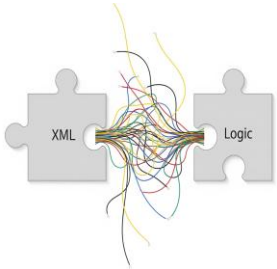


## La Metáfora de la Silla

Describe el resultado final: **"Quiero estar sentado"**. Compose gestiona la ejecución del estado.

# Separación de Archivos y Sincronización Manual



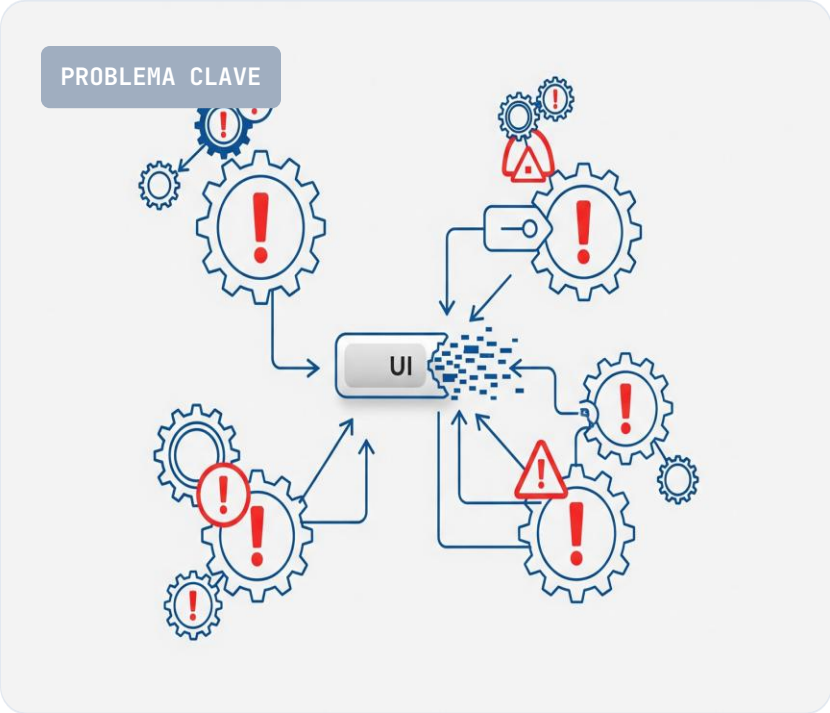

En el paradigma tradicional, la **UI se define en XML** y la **lógica en Java**.

El programador debe buscar referencias con `findViewById`, mantener variables de estado y actualizar manualmente la vista.

*"Si olvidamos llamar a `setText()`, el dato cambia pero la UI no".*

Este modelo es **propenso a errores de sincronización** y duplicación de lógica.

# Riesgo de Desincronización




Cuando el estado cambia en **varios lugares**, cada punto debe actualizar la UI manualmente.

Si uno se omite mediante `setText()` o `setVisibility()`, la interfaz queda **desactualizada** respecto a la lógica.

En aplicaciones grandes, esto genera bugs difíciles de rastrear y mantener debido a la **dispersión de la fuente de verdad**.

**Efecto Cascada:**

Múltiples controladores → Actualizaciones Manuales → Inconsistencia Visual.

# La UI Como Función del Estado

KOTLIN + COMPOSE

```kotlin
@Composable
fun CounterScreen() {
  var count by remember {
    mutableStateOf(0)
  }

  Button(onClick = { count++ }) {
    Text("El texto es ${count}")
  }
}
```

Compose integra UI y lógica en una **misma función**. La interfaz se describe de forma declarativa.

Cuando el estado cambia, Compose detecta el cambio y **redibuja solo lo necesario** (Recomposición).

*"La UI siempre reflejará el valor actual del estado, eliminando la desincronización manual."*

# Cómo Compose Mantiene el Estado

**RECUERDO**

`remember`
Conserva el valor entre recomposiciones.

**OBSERVADOR**

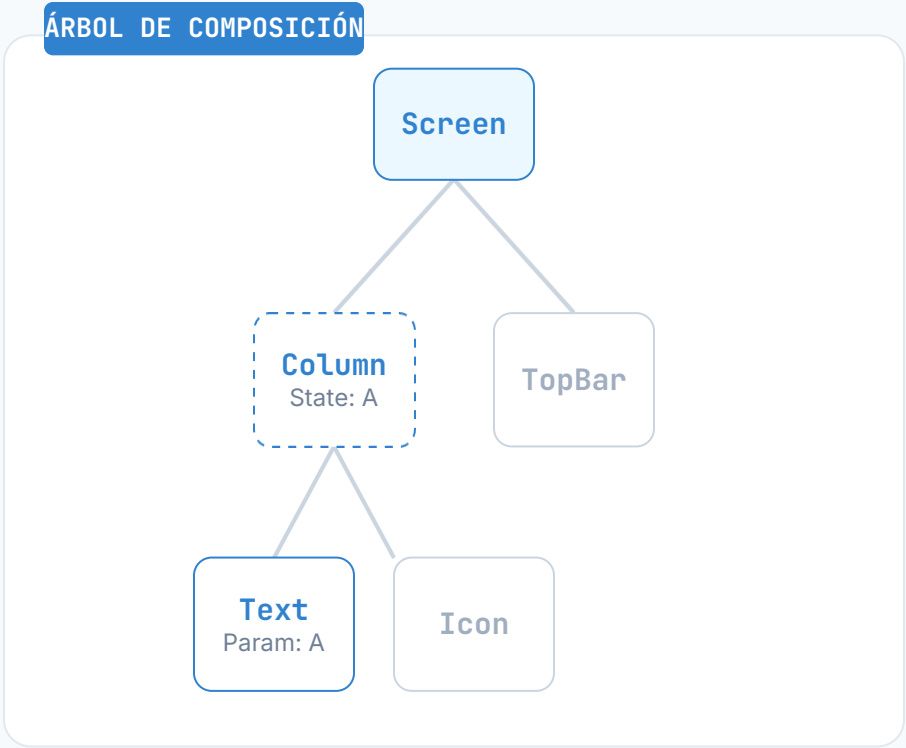
`mutableStateOf`
Notifica cambios para redibujar la UI.

**RESULTADO**

`Interfaz UI`
Se deriva automáticamente del estado.

- `@Composable` indica que la función genera elementos visuales dinámicos.
- El estado es la única fuente de verdad; no se requieren actualizaciones manuales.
- **Sin setters manuales:** Al cambiar el estado, la UI se sincroniza por sí sola.

# La Estructura Interna de Compose




Compose construye un **árbol jerárquico** que representa la estructura de las funciones Composable.

Cada nodo almacena:
- Los **parámetros** recibidos.
- El **estado** que observa.

Esta metadata permite la **Recomposición Inteligente**: solo se redibujan los nodos cuyo estado o parámetros han cambiado, optimizando el rendimiento al máximo.

```
@Composable
fun CounterScreen() {
    var count by remember { mutableStateOf(

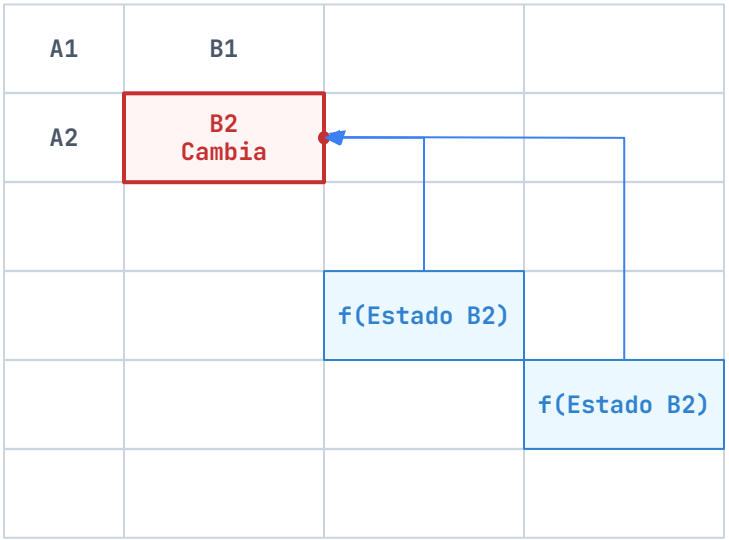
    Button(onClick = { count++ }) {
        Text("El texto es ${count}")
    }
}
```

```
Árbol de Composición
--------------------

CounterScreen
 └── Button
      └── Text("El texto es ${count}")
```

# Redibujar Solo lo Necesario

ANALOGÍA




Cuando un **estado cambia**, Compose no destruye toda la pantalla. Solo recompone los nodos que **dependen de ese estado**.

Lo podemos comparar con Excel:

*"Si cambias el valor de una celda, solo se recalculan las fórmulas que dependen de ella".*

# Qué Cambia y Qué se Mantiene

EXAMPLE: WEATHERDASHBOARD

```
@Composable
fun WeatherDashboard(temp: Int) {
  // Este nodo se ignora si solo cambia
  'temp'
  Text("Ciudad: Quito")

  // Este nodo se recompone siempre
  Text("Temperatura actual: $temp")

  if (temp > 30) {
    AlertIcon() // UI Dinámica
  }
}
```

ESTÁTICO

DINÁMICO

**Text("Ciudad: Quito")** es estático: no depende de *temp*, por lo que Compose lo omite en recomposiciones.

**Text("Temperatura actual...")** sí depende del estado y se redibuja automáticamente.

El bloque condicional **if (temp > 30)** crea UI dinámica basada en el estado, insertando o removiendo nodos del árbol.

# Ejemplos de árboles de recomposición

### Ejemplo 1

```
Column {
    Text("Título")
    Button(onClick = { count++ }) {
        Text("El texto es ${count}")
    }
}
```

```
CounterScreen
└── Column
      ├── Text("Título")
      └── Button
            └── Text("El texto es ${count}"
```

### Ejemplo 2

```
Column {
    Text("Título")
    Text("Subtítulo")
    Button(onClick = { count++ }) {
        Text("El texto es ${count}")
    }
    Text("Pie de página")
}
```

```
CounterScreen
└── Column
      ├── Text("Título")
      ├── Text("Subtítulo")
      ├── Button
      │     └── Text("El texto es ${count}"
      └── Text("Pie de página")
```

### Ejemplo 3

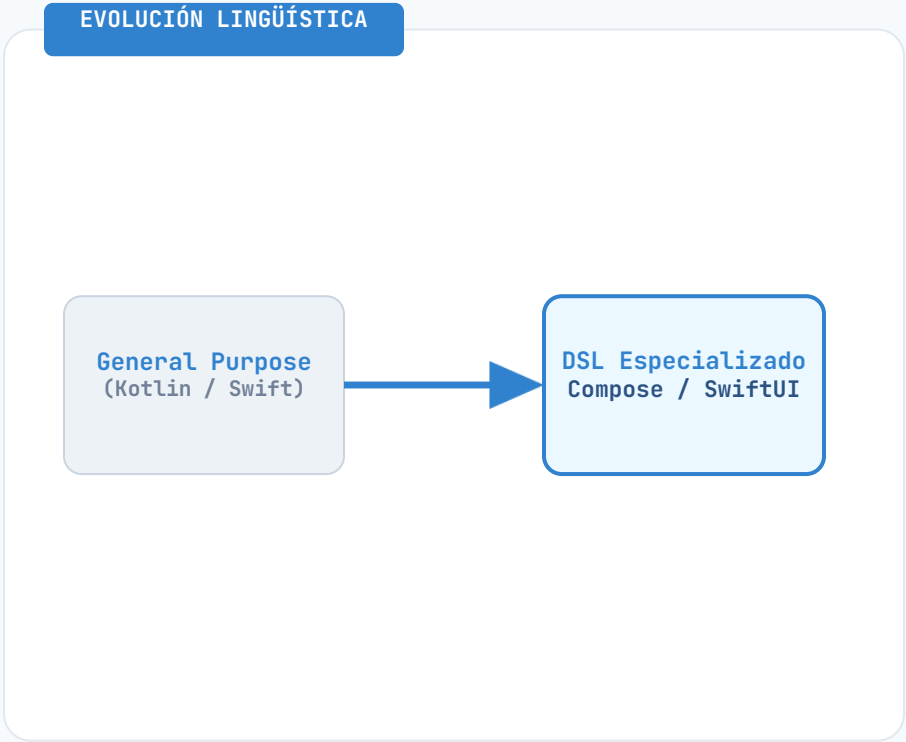
```
Column {
    Text("Título")

    Row {
        Text("A")
        Text("B")
        Text("C")
    }

    Button(onClick = { count++ }) {
        Text("El texto es ${count}")
    }
}
```

```
CounterScreen
└── Column
      ├── Text("Título")
      ├── Row
      │     ├── Text("A")
      │     ├── Text("B")
      │     └── Text("C")
      └── Button
            └── Text("El texto es ${count}"
```

# Lenguajes Específicos de Dominio




- Un DSL es un **"lenguaje dentro de un lenguaje"** diseñado para una tarea concreta.
- Compose y SwiftUI son DSL que **describen pantallas** de forma declarativa.
- **No se escriben algoritmos complejos:** se describe *qué* se quiere ver, no *cómo* construirlo paso a paso.

*"El código se convierte en la documentación visual de la interfaz."*

# Convergencia de la Industria Móvil

TABLA COMPARATIVA

| Concepto | Compose | SwiftUI |
|---|---|---|
| Vertical | `Column` | `VStack` |
| Horizontal | `Row` | `HStack` |
| Estilo | `Modifier` | `.padding()` |
| Estado | `remember` | `@State` |

- Ambos frameworks usan **contenedores verticales y horizontales** para estructurar la UI.
- Los **modificadores** funcionan como “adjetivos” que definen la apariencia de la interfaz.
- El estado local se maneja con **remember** en Compose y **@State** en SwiftUI.

*"Ambos frameworks comparten la filosofía declarativa moderna, unificando el desarrollo móvil."*

# Los Pilares del Diseño Declarativo

CONCEPTOS FUNDAMENTALES

### Contenedores

Reemplazan a `LinearLayout` y organizan la UI por bloques lógicos estructurados.

### Modificadores

Permiten ajustar tamaño, color y espaciado; *su orden de aplicación importa* en el resultado.

### Estado

Es la "memoria" del componente que persiste y reacciona durante las recomposiciones.

### Punto de Entrada

`@Composable` marca funciones que generan UI y habilita el motor declarativo.

# Diagnóstico de Rendimiento en Listas

ACTIVIDAD PRÁCTICA



## Escenario de Optimización

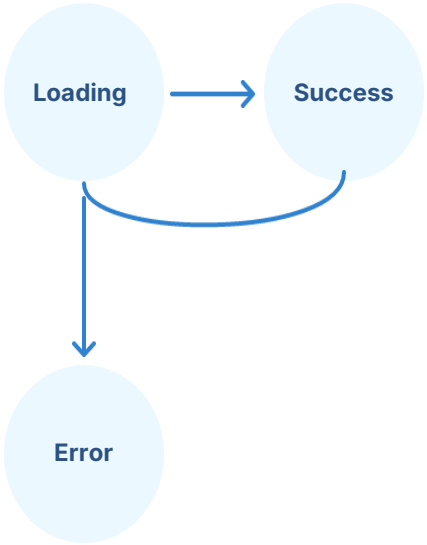
Un estudiante nota que su lista de **1000 elementos** se ralentiza significativamente durante el scroll.

❌ **El Problema:** Uso de `Column` con `forEach`. Esto obliga al sistema a componer y mantener en memoria la totalidad de los elementos simultáneamente.

✅ **La Solución:** Implementar `LazyColumn`, el equivalente declarativo de `RecyclerView`, que solo compone los elementos visibles en pantalla.

# Interfaces Reactivas Basadas en Estados

ACTIVIDAD PRÁCTICA




## Patrón UiState Robustos

Se define un `UiState` con variantes: **Loading**, **Success**, **Error** para modelar el estado real de la pantalla.

- Un bloque `when(uiState)` permite mostrar la UI correspondiente de forma segura y exhaustiva.
- Reduce errores de lógica al evitar estados inconsistentes en la interfaz.
- Este patrón es **robusto y escalable** para aplicaciones reales de alta complejidad.

# Conclusiones

RESUMEN DE CIERRE



## Evolución del Desarrollo Declarativo

- **Compose redefine** la forma de construir interfaces móviles mediante componentes reactivos.
- La UI se convierte en una **función pura del estado**, eliminando inconsistencias visuales.
- La **recomposición inteligente** optimiza el rendimiento y mejora la claridad del código.
- La convergencia con **SwiftUI** confirma la dirección de la industria hacia DSLs declarativos.